National University of Computer & Emerging Sciences

Lecture 9

Process Creation

The fork() system call

At the end of the system call there is a new process waiting to run once the scheduler chooses it

- A new data structure is allocated
- The new process is called the child process.
- The existing process is called the parent process.
- The parent gets the child's pid returned to it.
- The child gets 0 returned to it.
- Both parent and child execute at the same point after fork() returns

But we want the child process to do something else…

```
int pid;
int status = 0;

pid = fork();
if (pid>0) {
      /* parent */
      ……
      pid = wait(&status);
} else {
      /* child */
      ……
      exit(status);
}
```

*The **fork** syscall returns a zero to the child and the child process ID to the parent*

*Parent uses **wait** to slee… chil… retu… and…*

***Fork** creates an exact copy of the parent process*

***Wait** variants allow wait on a specific child or*

*Child process passes status back to parent on **exit**, to report success/failure*

Child Process Inherits

- Stack
- Memory
- Environment
- Open file descriptors.
- Current working directory
- Resource limits
- Root directory

Child process DOESNOT Inherit

- Process ID
- Different parent process ID
- Process times
- Own copy of files
- Resource utilization (initialized to zero)

The wait() System Call

- A child program returns a value to the parent, so the parent must arrange to receive that value
- The wait() system call serves this purpose
  - pid_t wait(int *status)
  - it puts the parent to sleep waiting for a child's result
  - when a child calls exit(), the OS unblocks the parent and returns the value passed by exit() as a result of the wait call (along with the pid of the child)
  - if there are no children alive, wait() returns immediately
  - also, if there are zombies, wait() returns one of the values immediately (and deallocates the zombie)

What is an orphan child?

- If the parent terminates without calling wait(), the child is adopted by init.

The solution is:

- Ensure that your parent process calls wait() or waitpid or etc, for every child process that terminates.

What is a zombie?

- In the interval between the child terminating and the parent calling wait(), the child is said to be a ‘zombie’.
- Even though its not running its taking up an entry in the process table.
- The process table has a limited number of entries.

exit()

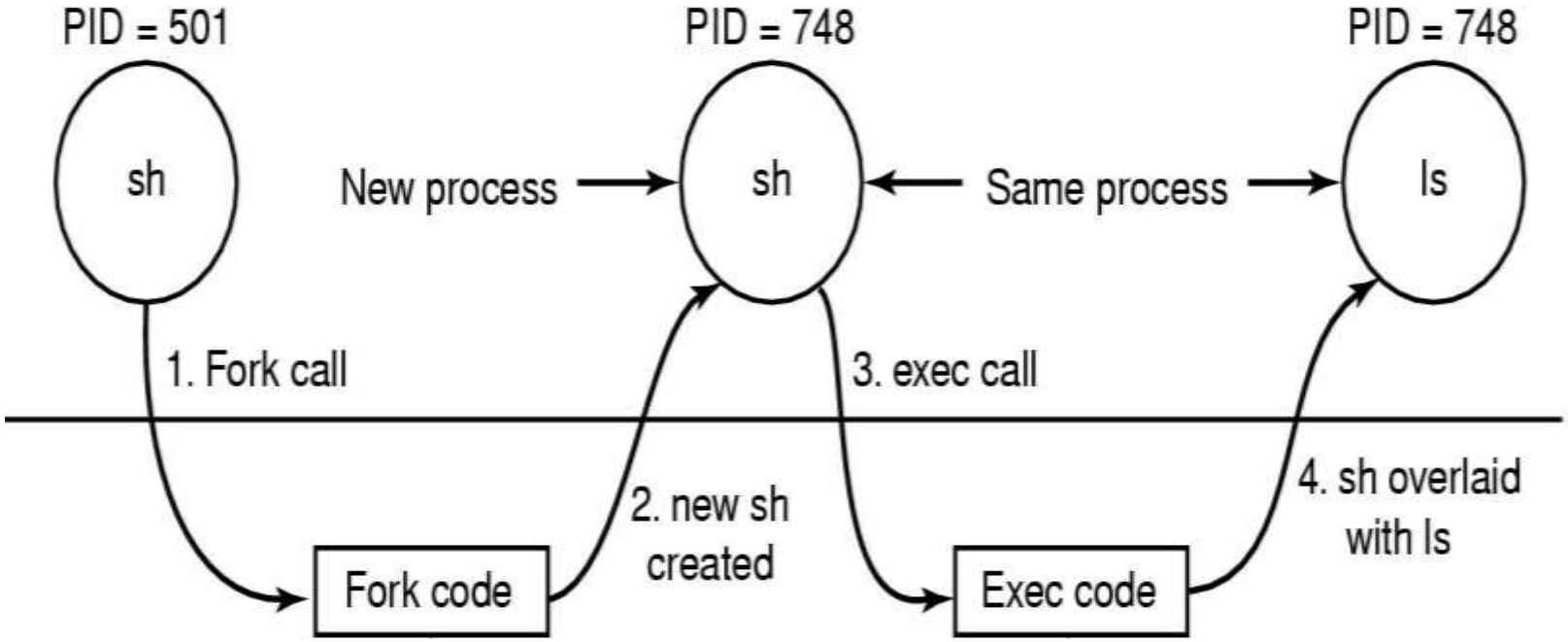
void exit(int *status***);**

- After the program finishes execution, it calls *exit*()
- This system call:
  - takes the “result” of the program as an argument
  - closes all open files, connections, etc.
  - deallocates memory
  - deallocates most of the OS structures supporting the process
  - checks if parent is alive:
    - If so, it holds the result value until parent requests it, process does not really die, but it enters the zombie/defunct state
    - If not, it deallocates all data structures, the process is dead

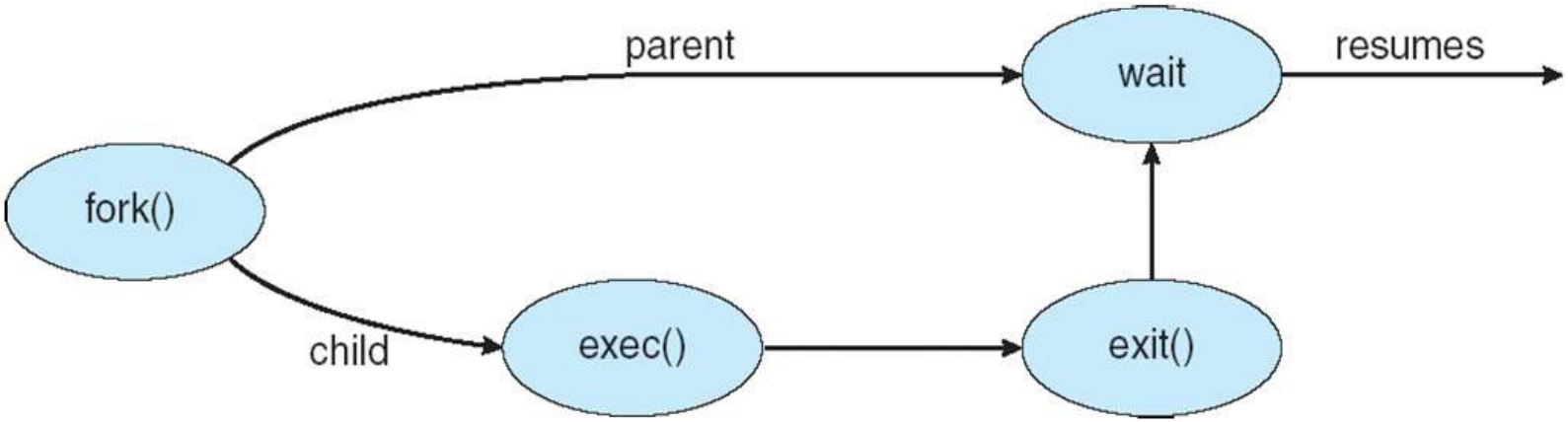
exec()

- We usually want the child process to run some other executable
- For Example, *ls - list directory contents*
- Overlays, performed by the family of exec system calls, can change the executing program, but can not create new processes.

The *ls* Command



Steps in executing the command *ls* type to the shell


parent
wait
resumes
fork()
child
exec()
exit()

execv

- int execv(const char *path, char *const argv[]);
- the current process image with a new process image.
- **path** is the filename to be executed by the child process
- When a C-language program is executed as a result of this call, it is entered as a C-language function call as follows:
  - int main (int argc, char *argv[]);
- The **argv** array is terminated by a null pointer.
- The null pointer terminating the **argv** array is not counted in **argc**.

```
main()
 {
    pid_t pid;
    char *const parmList[] = {"/bin/ls", "-l", "/u/userid/dirname", NULL};

    if ((pid = fork()) == -1)
       perror("fork error");
    else if (pid == 0) {
       execv("/bin/ls", parmList);
       printf("Return not expected. Must be an execv error.n");
    }
 }
```

How can a parent and child process communicate?

- Through any of the normal IPC mechanism schemes.
- But have special ways to communicate
  - For example
    - The variables are replicas
    - The parent receives the exit status of the child

How can a parent and child process communicate?

- Through any of the normal IPC mechanism schemes.
- But have special ways to communicate
  - For example
    - The variables are replicas
    - The parent receives the exit status of the child